Instructivo de Práctica — Sesión 3: Tema, estados y accesibilidad en ExploraEC

> 📌 **Qué vas a entregar:** al terminar esta práctica, ExploraEC va a tener el tema de marca del curso aplicado, los tres estados de carga/vacío/error funcionando en la pantalla de Inicio, un layout que se adapta al ancho de pantalla, y una primera pasada de accesibilidad — todo sobre las pantallas que ya construiste en la Sesión 2, sin agregar ninguna pantalla nueva.

Objetivo

Aplicar un tema Material consistente a ExploraEC, construir y conectar los tres estados que toda pantalla con datos necesita (carga, vacío, error), adaptar el layout de Inicio al ancho de pantalla disponible, y aplicar una pasada básica de accesibilidad.

Requisitos previos

- Proyecto `exploraec` de la Sesión 2 funcionando: modelo `Place`, `PlaceCard`, Inicio/Detalle/Formulario, navegación inferior.
- Repositorio de la Sesión 2 subido a GitHub, con el commit correspondiente.
- Emulador Android iniciado (creado en la Sesión 1). Si está cerrado, iniciarlo antes de empezar y esperar a que cargue el escritorio de Android por completo:

`bash flutter emulators # lista los emuladores; copia el id flutter emulators --launch <id> flutter devices # debe aparecer un dispositivo Android`

Si `flutter devices` muestra también Chrome o el escritorio, ejecutar siempre con `flutter run -d <id-del-dispositivo-android>`.

Tiempo estimado

Total: 55 minutos

Paso	Descripción	Tiempo
1	Aplicar el tema de marca	8 min
2	Verificar tipografía y espaciado centralizados	7 min
3	Conectar los 3 estados a Inicio	20 min
4	Layout responsivo con `LayoutBuilder`	10 min
5	Accesibilidad básica con `Semantics`	10 min
6	*(Opcional)* Modo oscuro	5 min, fuera de los 55

Punto de partida

```
git fetch starter
git checkout starter/sesion-03 -- lib pubspec.yaml PLACEHOLDERS.md
flutter pub get
```

Esto trae sobre tu propio proyecto `exploraec` la carpeta `lib/theme/` completa, los archivos nuevos `lib/widgets/loading_view.dart`, `lib/widgets/empty_view.dart`, `lib/widgets/error_view.dart`, y las versiones actualizadas de `lib/main.dart`, `lib/models/place.dart`, `lib/screens/home_screen.dart` y `lib/widgets/place_card.dart` — sin tocar `android/` / `ios/`. Revisa `PLACEHOLDERS.md` (recién llegado a la raíz de tu proyecto) para el detalle exacto de qué bloque corresponde a cada paso, con el comentario `// Por qué:` de cada uno.

Paso 1 — Aplicar el tema de marca (8 min)

Por qué se hace

`AppTheme.theme` centraliza colores, tipografía y estilos de componente en un solo lugar — activarlo es el cambio de mayor impacto visual con menor esfuerzo de todo el curso hasta ahora.

Instrucciones

1. Abrir `lib/main.dart`.
2. Localizar el bloque `// TODO(sesion-03): ... (Paso 1 — aplicar el tema)`.
3. Borrar la línea activa `theme: ThemeData(useMaterial3: true),` y descomentar `theme: AppTheme.theme,`.
4. Guardar y confirmar hot reload.

Resultado esperado

La `AppBar` cambia a fondo navy con texto blanco; los botones y elementos interactivos toman el color teal; las tarjetas muestran esquinas más redondeadas.

Errores comunes

- **`Undefined name 'AppTheme'`**. *Solución:* confirmar que se copió `lib/theme/app_theme.dart` dentro del proyecto y que el `import '../theme/app_theme.dart';` (o `theme/app_theme.dart` según la carpeta) está presente en `main.dart`.
- **El tema no cambia nada visualmente**. *Solución:* confirmar que se borró la línea `theme: ThemeData(useMaterial3: true)` — dejar ambas líneas activas es un error de sintaxis de Dart (dos valores para el mismo parámetro nombrado).

✅ Antes de continuar, confirma que...

- ☐ La `AppBar` de Inicio se ve con fondo navy y texto blanco.
- ☐ El botón "Guardar" del formulario se ve en color teal.

Paso 2 — Verificar tipografía y espaciado ya centralizados (7 min)

Por qué se hace

`PlaceCard` ya fue escrita (Sesión 2, con ajustes de la Sesión 3) para usar `AppSpacing` y `Theme.of(context).textTheme` en vez de números y estilos sueltos — este paso es de lectura y verificación, no de escritura, para entender el patrón antes de aplicarlo por cuenta propia en la Tarea o el proyecto final.

Instrucciones

1. Abrir `lib/theme/app_theme.dart` y ubicar la clase `AppSpacing` (constantes `xs`, `sm`, `md`,

`lg`, `xl`) y el `TextTheme` dentro de `AppTheme.theme`.

2. Abrir `lib/widgets/place_card.dart` y ubicar los usos de `AppSpacing.md`, `AppSpacing.sm`, `AppSpacing.xs` y de `Theme.of(context).textTheme`.
3. Cambiar temporalmente `AppSpacing.md` a un valor mucho más grande (por ejemplo `40`) directamente en `app_theme.dart`, guardar, y observar cómo cambia el espaciado de **todas** las tarjetas a la vez. Revertir el cambio a `12` antes de continuar.

Resultado esperado

Al cambiar `AppSpacing.md`, el espaciado interno de cada `PlaceCard` cambia simultáneamente, sin haber tocado `place_card.dart`.

Errores comunes

- **El cambio no se refleja**. *Solución:* confirmar que se guardó el archivo y que el hot reload se disparó (`r` en la terminal si no fue automático).

✅ Antes de continuar, confirma que...

- ☐ Se identificó dónde vive `AppSpacing` y se comprobó su efecto en cascada sobre las tarjetas.
- ☐ El valor de `AppSpacing.md` quedó de nuevo en `12` antes de seguir.

Paso 3 — Conectar los 3 estados a Inicio (20 min)

Por qué se hace

Sin estos tres estados, Inicio no tiene forma de comunicar si está esperando datos, si no hay ninguno, o si algo salió mal — el mínimo de feedback visual esperado de cualquier pantalla del curso desde hoy en adelante.

Instrucciones

1. Abrir `lib/models/place.dart`, localizar el bloque `// TODO(sesion-03): ... (Paso 3 — carga simulada)`, borrar la versión activa de `fetchLugaresSimulado` y descomentar la versión con `Future.delayed` y los parámetros `forzarError` / `forzarVacio`.
2. Abrir `lib/screens/home_screen.dart`, localizar el bloque `// TODO(sesion-03): ... (Paso 3 — estados loading/vacío/error)`, borrar `body: const Center(child: Text('Cargando lugares...'))` y descomentar el `body: FutureBuilder<List<Place>>(...)` completo.
3. Ejecutar `flutter run` (o hot reload si ya estaba corriendo) y observar el estado de carga

(aparece brevemente al iniciar, por el `Future.delayed` de 1 segundo).

4. Tocar el ícono " ⋮ " de la `AppBar` → "Simular: vacío" → confirmar que aparece `EmptyView`.
5. Tocar " ⋮ " → "Simular: error" → confirmar que aparece `ErrorView` con el botón "Reintentar".
6. Presionar "Reintentar" con el modo todavía en "error" → confirma que vuelve a mostrar el error (comportamiento esperado, ya que el modo simulado sigue activo). Cambiar a "Simular: normal" y confirmar que ahora sí carga la lista.

Resultado esperado

Los tres estados se ven correctamente al cambiarlos desde el menú " ⋮ ": un `CircularProgressIndicator` con mensaje, una pantalla de "no hay lugares" con ícono, y una pantalla de error con botón de reintentar.

Errores comunes

- **`type 'Null' is not a subtype of type 'List<Place>'`**. *Solución:* revisar que el `FutureBuilder` comprueba `ConnectionState.waiting` y `snapshot.hasError` **antes** de leer `snapshot.data` — leerlo antes de esas comprobaciones puede encontrar `null`.
- **`EmptyView` nunca aparece**. *Solución:* confirmar que la comprobación `lugares.isEmpty` está después de la comprobación de error, no antes ni fusionada con ella.
- **El estado de carga nunca se alcanza a ver (muy rápido)**. *Solución:* es esperado con un `Future.delayed` de 1 segundo en una recarga rápida — no es un error, simplemente ocurre y termina muy rápido para notarlo sin prestar atención.

✅ Antes de continuar, confirma que...

- ☐ El estado "Simular: vacío" muestra `EmptyView`.
- ☐ El estado "Simular: error" muestra `ErrorView` con el botón "Reintentar" visible.
- ☐ El estado "Simular: normal" vuelve a mostrar la lista de 6 lugares.

Paso 4 — Layout responsivo con `LayoutBuilder` (10 min)

Por qué se hace

La misma lista de lugares debería aprovechar el espacio extra de una pantalla ancha (tablet, teléfono en horizontal) en vez de estirar una sola columna angosta sobre todo el ancho disponible.

Instrucciones

1. Abrir `lib/screens/home_screen.dart`, localizar el bloque `// TODO(sesion-03): ... (Paso 4 — layout responsivo)`, borrar la versión activa de `_buildLista` (el `ListView.builder` simple) y descomentar la versión con `LayoutBuilder`.
2. En la versión descomentada, localizar `childAspectRatio: 2.4,` y cambiarlo a `childAspectRatio: 2.2,`. Con `2.4` la tarjeta queda unos 5-6 píxeles demasiado baja en una celda de ~300 px de ancho, y aparece la franja de `RenderFlex overflowed` justo después de cruzar los 600 px y los 900 px de ancho. Con `2.2` la celda tiene alto suficiente en todo el rango.
3. Con la app corriendo en el emulador, rotar a horizontal (`Ctrl+F11` / `Ctrl+F12` en el emulador Android, según el sistema) o redimensionar la ventana si se corre con `flutter run -d chrome`.
4. Observar el cambio de `ListView` (una columna) a `GridView` (2-3 columnas) al cruzar los 600 píxeles lógicos de ancho.

Resultado esperado

En vertical (angosto), Inicio se ve igual que antes (una columna). En horizontal o en una ventana ancha, Inicio muestra 2-3 columnas de tarjetas.

Errores comunes

- **`A RenderFlex overflowed by N pixels`** al pasar a `GridView`. *Solución:* confirmar que `childAspectRatio` vale `2.2` (con `2.4` la tarjeta desborda 5-6 px en celdas de ~300 px, es decir, justo al pasar los 600 y los 900 px de ancho; con valores mayores, peor). Bajar el valor da más alto a la celda — si el texto de la descripción es muy largo, confirmar que sigue con `maxLines: 2` y `overflow: TextOverflow.ellipsis` en `PlaceCard`.
- **No hay forma de rotar el emulador**. *Solución:* si no hay emulador o dispositivo a mano para probar el modo ancho, ejecutar con `flutter run -d chrome` y redimensionar manualmente la ventana del navegador por debajo/encima de 600px — es una alternativa válida para esta verificación puntual.

✅ Antes de continuar, confirma que...

- ☐ En orientación vertical (o ventana angosta), Inicio se ve como una lista de una columna.
- ☐ En orientación horizontal (o ventana ancha), Inicio se ve como una grilla de 2 o más columnas.
- ☐ `childAspectRatio` vale `2.2` y no aparece ninguna franja de overflow al pasar por 600 px y 900 px.

Paso 5 — Accesibilidad básica con `Semantics` (10 min)

Por qué se hace

Sin una etiqueta semántica explícita, un lector de pantalla recorre el contenido de `PlaceCard` como textos sueltos sin relación, en vez de anunciarla como una sola tarjeta interactiva con su nombre y categoría.

Instrucciones

1. Abrir `lib/widgets/place_card.dart`, localizar el bloque `// TODO(sesion-03): ... (Paso 5 – accesibilidad)`, borrar `child: _buildContenido(context)` activo y descomentar la versión con `Semantics(...)`.
2. Activar un lector de pantalla para verificar: - **Android:** Ajustes → Accesibilidad → TalkBack → activar. Tocar una tarjeta y escuchar el anuncio. - **Alternativa sin lector de pantalla:** en VS Code/Android Studio, abrir el "Flutter Inspector" → pestaña de accesibilidad (Semantics Tree/Debug Paint) y confirmar que el nodo de la tarjeta muestra el `label` esperado.
3. Verificar el `Semantics` que acabas de descomentar: debe tener exactamente estas cuatro propiedades — `label`, `hint`, `button: true` y **`excludeSemantics: true`**. Si falta la última, agrégala debajo de `button: true,`:

`dart child: Semantics( label: '${place.nombre}, categoría ${place.categoria}', hint: 'Toca dos veces para ver el detalle', button: true, excludeSemantics: true, child: _buildContenido(context), ),`

Sin `excludeSemantics: true`, el lector de pantalla anuncia el `label` y después repite nombre, categoría y descripción como textos sueltos. 4. Desactivar TalkBack al terminar (puede hacer más lento el resto de la práctica si queda activo).

Resultado esperado

El lector de pantalla (o el inspector de accesibilidad) anuncia/muestra el nombre y la categoría del lugar como una sola unidad, identificada como interactiva (botón).

Errores comunes

- **TalkBack queda activado y complica navegar el resto del emulador**. *Solución:* desactivarlo apenas se termine de verificar este paso — vuelve a activarse desde el mismo menú de Ajustes.
- **El Inspector de Flutter no muestra el árbol de Semantics**. *Solución:* confirmar que el modo "Debug Paint"/"Semantics Debugger" está activado desde el propio Inspector, no solo abierto.

- **El lector de pantalla anuncia el `label` y después vuelve a leer el nombre, la categoría y la descripción por separado**. *Solución:* agregar `excludeSemantics: true,` debajo de `button: true,` en el `Semantics` (ver el paso 3 de arriba) — sin esa propiedad, además del `label` personalizado, se sigue leyendo el contenido de `_buildContenido()` como texto suelto.

✅ Antes de continuar, confirma que...

- ☐ `Semantics` incluye `excludeSemantics: true`.
- ☐ Se verificó, con TalkBack o con el Inspector, que `PlaceCard` anuncia/expone un `label` con el nombre y la categoría del lugar, sin repetir después nombre, categoría y descripción por separado.

Paso 6 (opcional) — Modo oscuro (5 min)

> Este paso **no cuenta dentro de los 55 minutos** ni bloquea ninguna sesión futura. Hazlo si terminaste antes que el resto o como repaso.

Por qué se hace

Un tema bien centralizado permite un segundo look completo sin tocar ninguna pantalla: `AppTheme.darkTheme` ya viene escrito dentro de `lib/theme/app_theme.dart` (misma semilla teal, `Brightness.dark`), y `PlaceCard`, `LoadingView` y `EmptyView` ya toman sus colores del `ColorScheme` en vez de teal o grises fijos. Solo falta decirle a `MaterialApp` que lo use.

Instrucciones

1. Abrir `lib/main.dart` y localizar el bloque `// TODO(sesion-03): OPCIONAL — ... (Paso 6 — modo oscuro)`, justo debajo de `theme: AppTheme.theme,`.
2. **Descomentar** las dos líneas (no hay nada que borrar):

```
dart darkTheme: AppTheme.darkTheme, themeMode: ThemeMode.system,
```

3. Guardar y hacer hot restart (`R` en la terminal de `flutter run`).
4. Cambiar el modo del sistema y observar que la app lo sigue: - **Emulador Android:** Ajustes → Pantalla → Tema oscuro (o el interruptor de "Tema oscuro" en los ajustes rápidos). Desde la terminal: `adb shell cmd uimode night yes` (y `night no` para volver al

claro). - **Chrome (`flutter run -d chrome`):** cambiar el tema del sistema operativo.

5. Con el modo oscuro activo, recorrer los tres estados desde el menú " ⋮ " (normal, vacío, error) y confirmar que todos se leen bien.

Resultado esperado

En modo oscuro, el fondo y las tarjetas son oscuros, el texto es claro, los pines y el indicador de carga toman un teal claro legible, y la `AppBar` conserva el navy de marca. En modo claro, la app se ve exactamente igual que en los pasos anteriores.

Errores comunes

- **No pasa nada al activar el tema oscuro del sistema**. *Solución:* confirmar que **ambas** líneas están descomentadas (`darkTheme` y `themeMode`) y hacer hot restart, no solo hot reload — `themeMode` se lee al construir el `MaterialApp` .
- **Texto ilegible (oscuro sobre oscuro) en alguna pantalla propia**. *Solución:* casi siempre es un color fijo (`Colors.grey` , `AppTheme.teal` , `Color(0xFF...)`). Reemplazarlo por un color de `Theme.of(context).colorScheme` (por ejemplo `onSurfaceVariant` para texto secundario, `primary` para acentos). Las pantallas de Mapa y Favoritos siguen siendo placeholders con gris fijo; se ven aceptables, pero se rehacen en sesiones posteriores.

✅ Antes de continuar, confirma que...

- ☐ Con el sistema en modo oscuro, ExploraEC se ve con fondo oscuro y texto claro.
- ☐ Con el sistema en modo claro, ExploraEC se ve igual que antes del Paso 6.

Entregable de la práctica

- Tema de marca aplicado en toda la app.
- Los tres estados (carga/vacío/error) funcionando y verificados en Inicio.
- Layout responsivo verificado (vertical y horizontal).
- `Semantics` aplicado en `PlaceCard` y verificado con TalkBack o el Inspector.
- *(Opcional)* Modo oscuro activado con `darkTheme` + `themeMode: ThemeMode.system` .

Conexión con la siguiente sesión

En la Sesión 4 se reemplaza el placeholder de la pestaña "Mapa" (hoy un simple texto "Próximamente") por un mapa real con `flutter_map` , mostrando la posición actual del

usuario y los lugares como marcadores. Es indispensable llegar con el tema y los tres estados de hoy funcionando: la Sesión 4 va a reutilizar `LoadingView` / `ErrorView` para el propio proceso de obtener la ubicación del usuario, con el mismo patrón visto hoy.